- Implement the **Repository Pattern** in Laravel 12 using **Query Builder**.
- Create an interface and concrete class for data access.
- Bind the interface to the implementation using Laravel's service container.
- Inject the repository into a controller and display the data in a **Bootstrap**-styled view.

This approach separates business logic from data access, improving your application's testability and maintainability.

11.10 Conclusion

In this chapter, we explored the **Query Builder** in Laravel 12, focusing on its capabilities for performing complex queries, joins, and aggregate functions. We also implemented the **Repository Pattern** to demonstrate how to structure your code for better maintainability and testability.

We built several hands-on labs, including:

- A **Car Management Application** using Query Builder for CRUD operations.
- A **Report Application** using joins and aggregate functions to summarize data.
- A **Repository Pattern** implementation to separate data access logic from business logic.

These exercises provided practical experience with Laravel's Query Builder, showcasing its flexibility and power in building robust applications. You also learned how to create a clean and user-friendly interface using **Bootstrap 5**.

You can now apply these concepts to your own projects, leveraging the power of Laravel's Query Builder to create efficient and maintainable applications.